

OPEN ARCHITECTURE STANDARD · v0.1.0

EASRA

Enterprise AI Systems Reference Architecture

The open standard for production-grade Enterprise AI.

VENDOR-NEUTRAL

No lock-in. Every layer maps to ≥ 2 clouds.

PRODUCTION-READY

Runtime, deployment & scaling
— not a slideware framework.

VERIFICATION-FIRST

Grounding, citation, factuality
baked in — verification $\neq$
evaluation.

CLOUD-AGNOSTIC

Azure · AWS · GCP · OSS-K8s —
same architecture, four impls.

Enterprise AI stacks are one-offs.

Every team is redesigning the same architecture — badly, in isolation, with different vocabulary.

No shared vocabulary

Teams argue about 'agent' vs 'workflow' vs 'orchestrator' before writing a line of code.

Security bolted on

Prompt injection, tool safety and PII treated as add-ons, not architectural concerns.

Verification confused with evaluation

Offline metrics ship to prod. No first-class verification of grounding, citation, factuality.

Cost & observability afterthoughts

Token spend, latency SLOs and tool traces retrofitted after the outage.

An open architecture standard for Enterprise AI.

Vendor-neutral. Production-grade. Standards-aligned.

Vendor-neutral

Every layer maps cleanly to Azure, AWS, GCP and open-source. No lock-in.

Security by design

Zero trust, four trust boundaries, prompt-injection resistance in the layers, not on top.

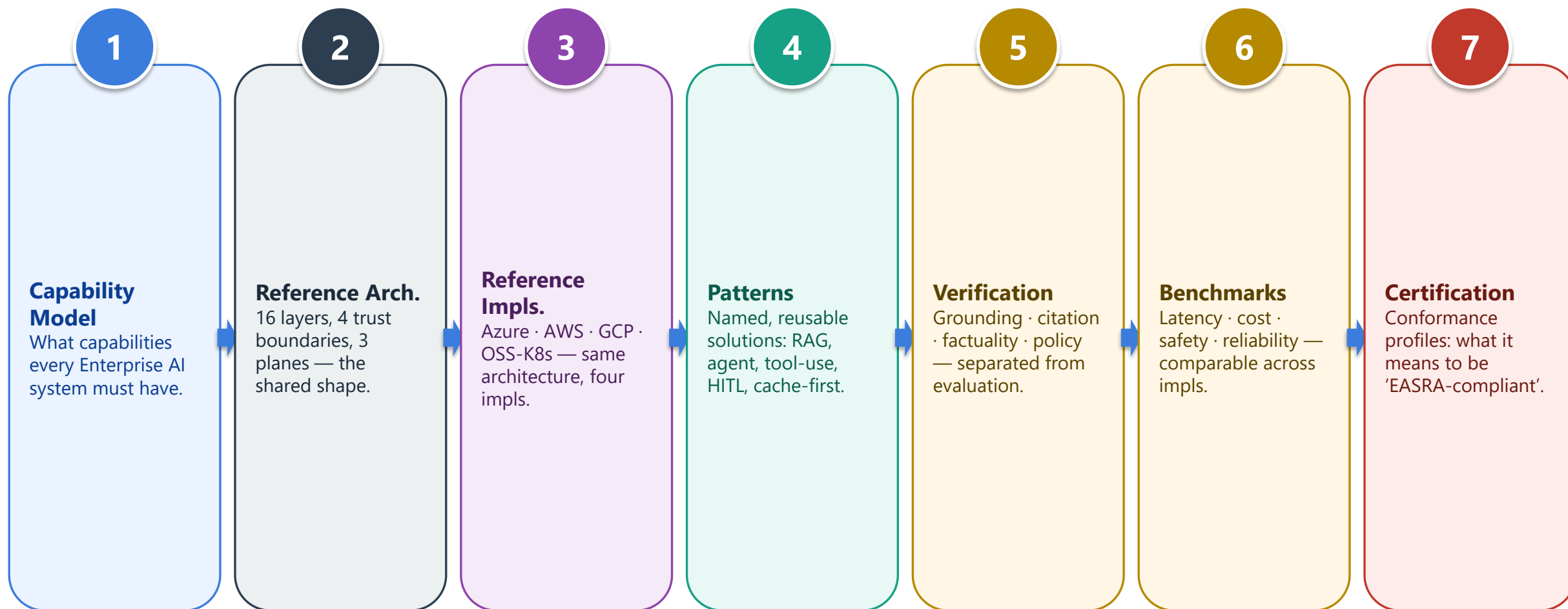
Verification-first

Grounding, citation, factuality, policy and safety verification separated from evaluation.

Cost + observability by default

Tokens, latency, cost, prompts, tools traced through OpenTelemetry from every component.

EASRA is not just a diagram. It is a full stack — like TOGAF, like OpenTelemetry.



Ladders 1–3 are v0.1.0 today. Ladders 4–7 are the roadmap.

P1

Vendor Neutrality

Logical layers; each maps to ≥ 2 independent implementations.

P2

Security by Design

Zero-trust, least-privilege, prompt-injection resistance are architectural.

P3

Verification by Design

Every AI output is verifiable; verification $\neq$ evaluation.

P4

Observability by Default

Traces, metrics, logs, token/cost accounting emitted from every component.

P5

Loose Coupling

Layers communicate only through published interfaces.

P6

Externalized State

Memory, sessions, caches live outside compute; compute is stateless.

P7

Human-in-the-Loop

Every autonomous action has escalation, override and audit paths.

P8

Failure Isolation

Layer failure degrades gracefully; no single layer cascades a full outage.

P9

Cost Awareness

Token, compute, storage cost are explicit design inputs.

P10

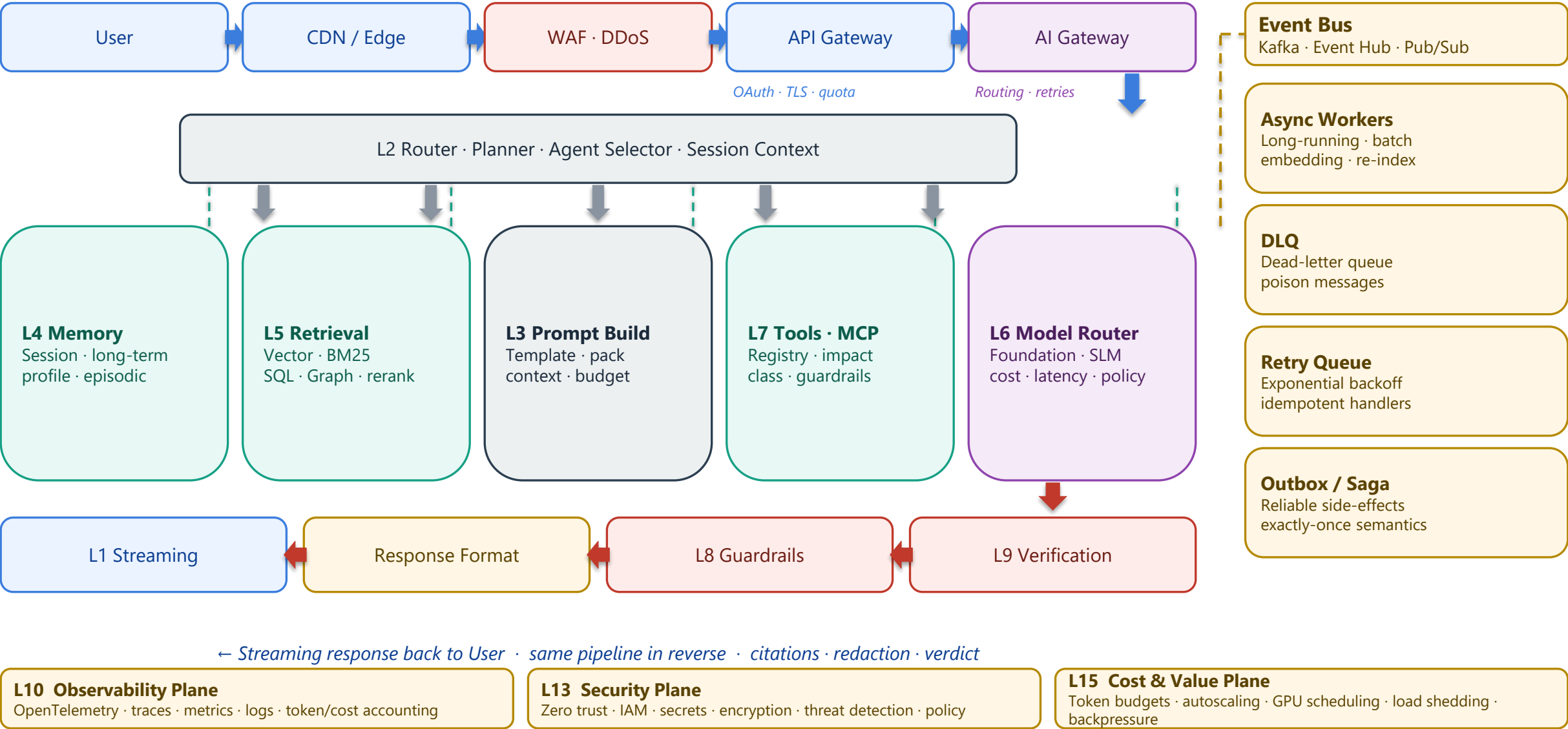
Evolvability

New models, tools, agents, channels added without redesigning the architecture.

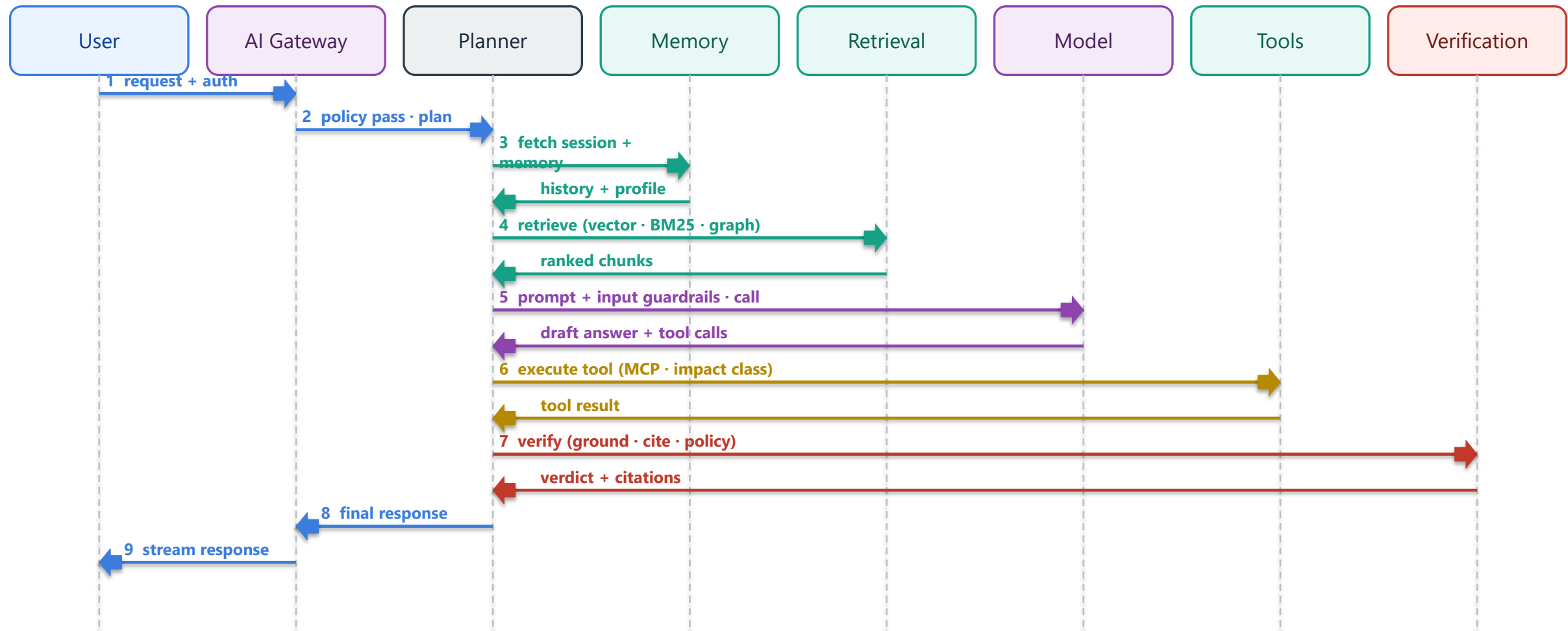
EASRA · The Sixteen Layers



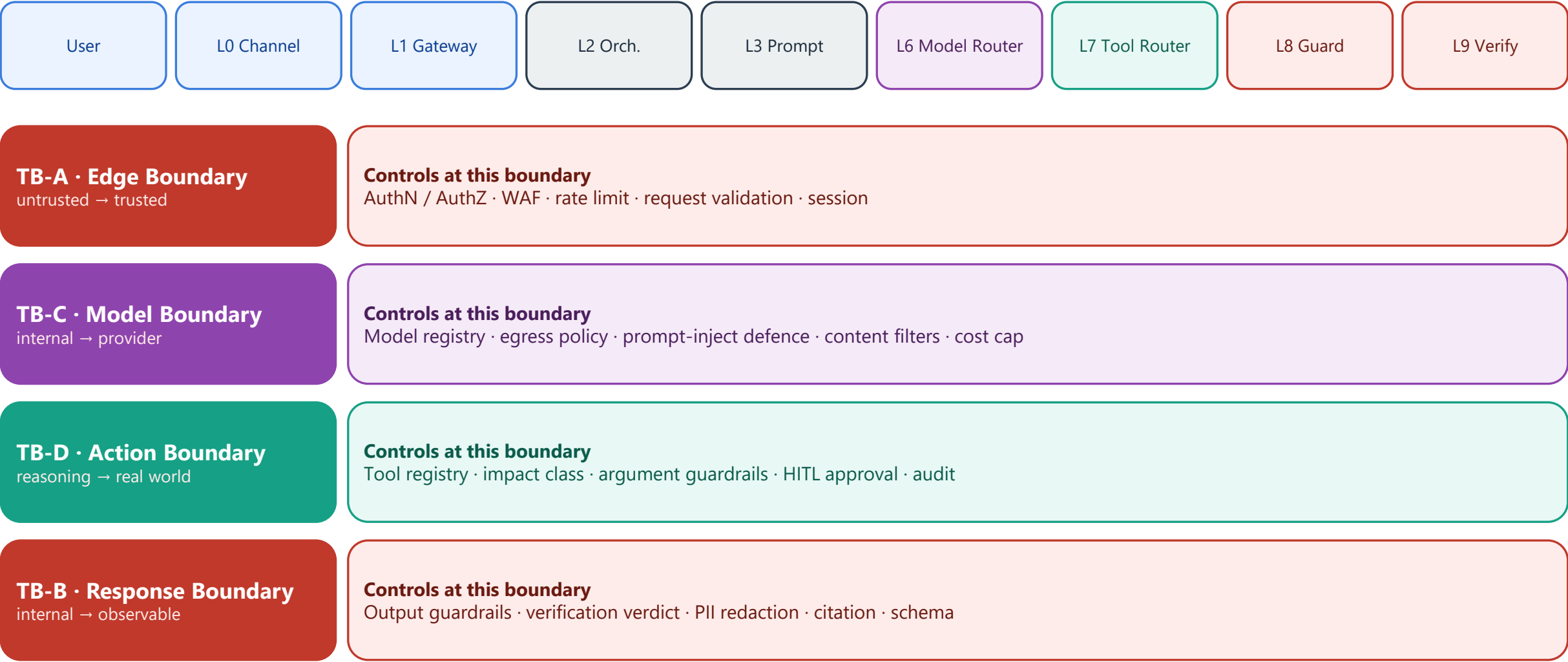
Request flows down through the pipeline; response streams back through verification and guardrails. Async work fans out to an event bus.



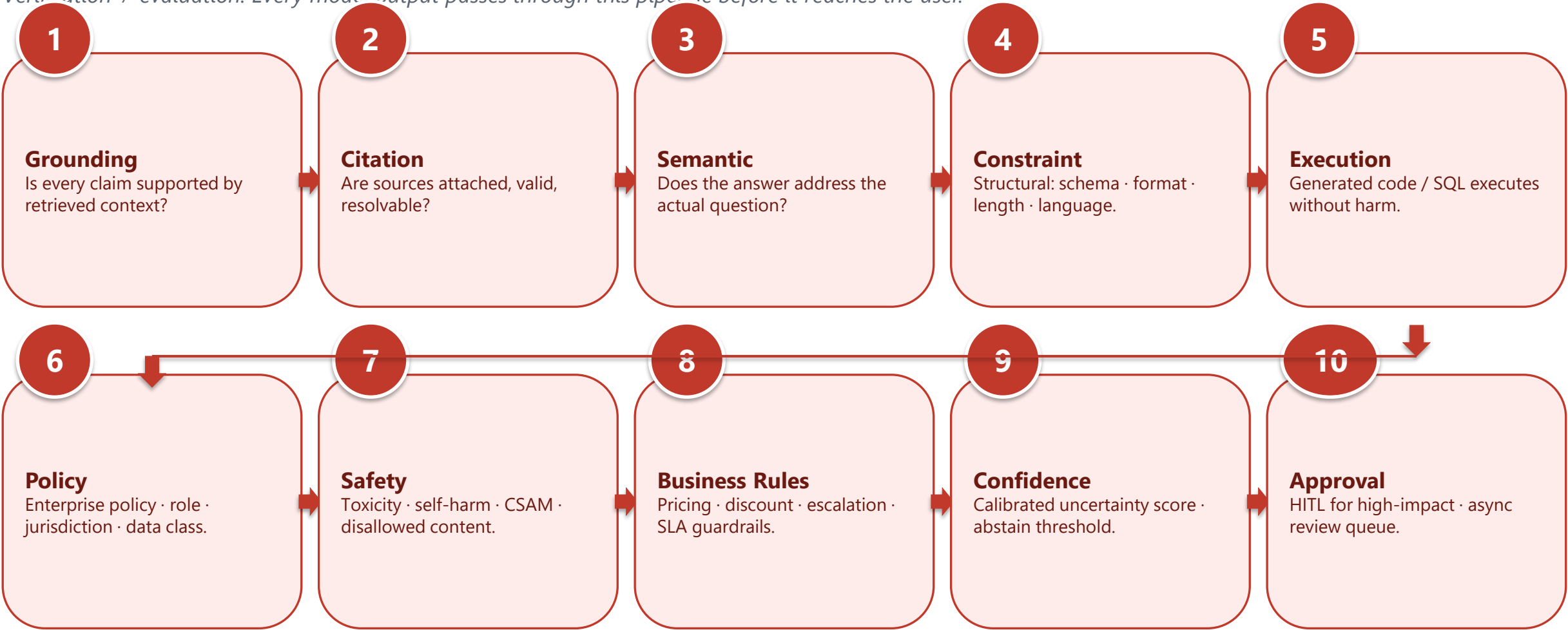
One request across the actors that actually cooperate at runtime. Read top-to-bottom; each arrow is a real hop with real latency.



Every request crosses four boundaries. Each has its own attackers, controls, and failure modes.



Verification ≠ evaluation. Every model output passes through this pipeline before it reaches the user.



Enterprise AI has three planes — just like Kubernetes. Confusing them is how governance, cost and reliability go wrong.

Control Plane

Decides · governs · configures

- Policy Engine (OPA · Cedar)
- Configuration store
- Model Registry
- Prompt Registry
- Workflow Registry
- Tool Registry
- Feature Flags
- Model routing rules
- Rate-limit + quota rules

Data Plane

Runs · executes · serves traffic

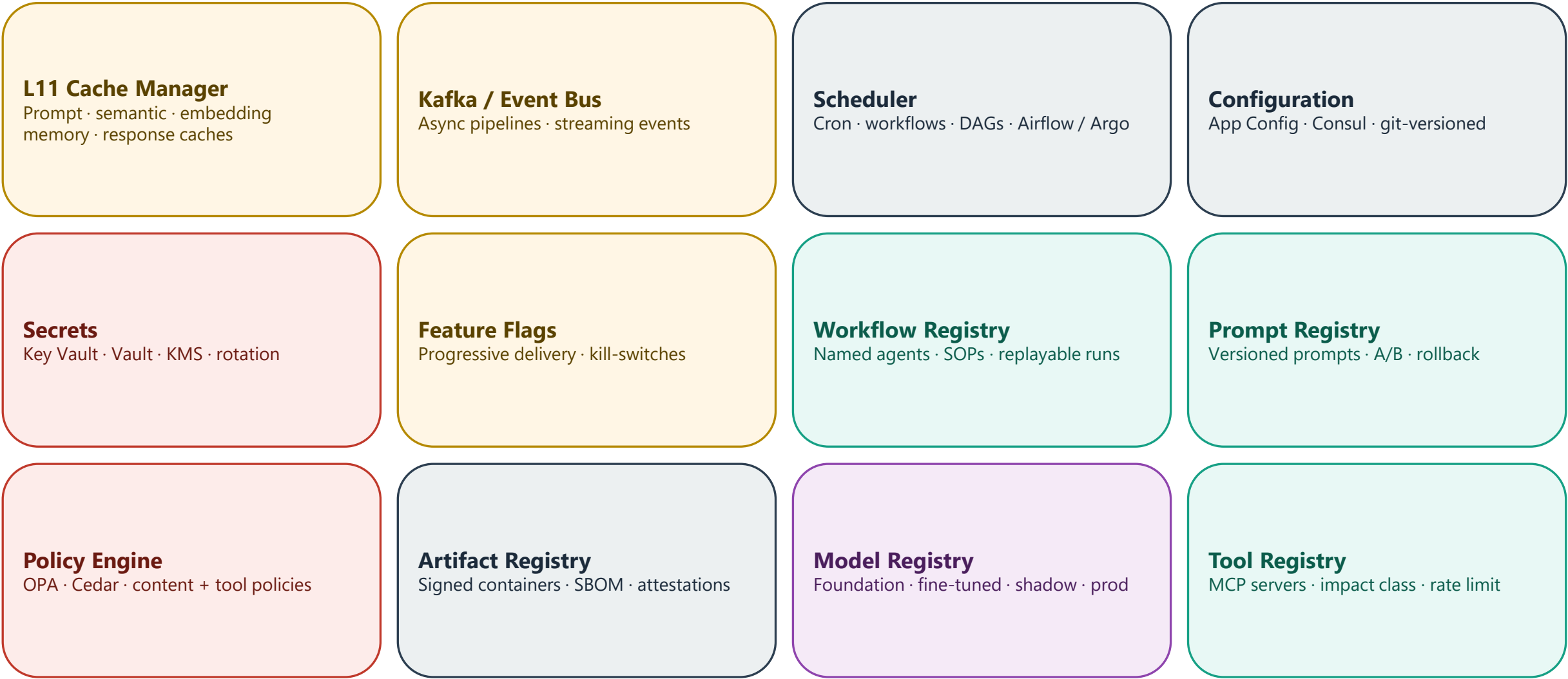
- Prompts & context assembly
- Embeddings & retrieval
- Model inference calls
- Tool executions (MCP)
- Verification checkers
- Guardrail evaluators
- Streaming responses
- Cache reads / writes
- Event bus messages

Management Plane

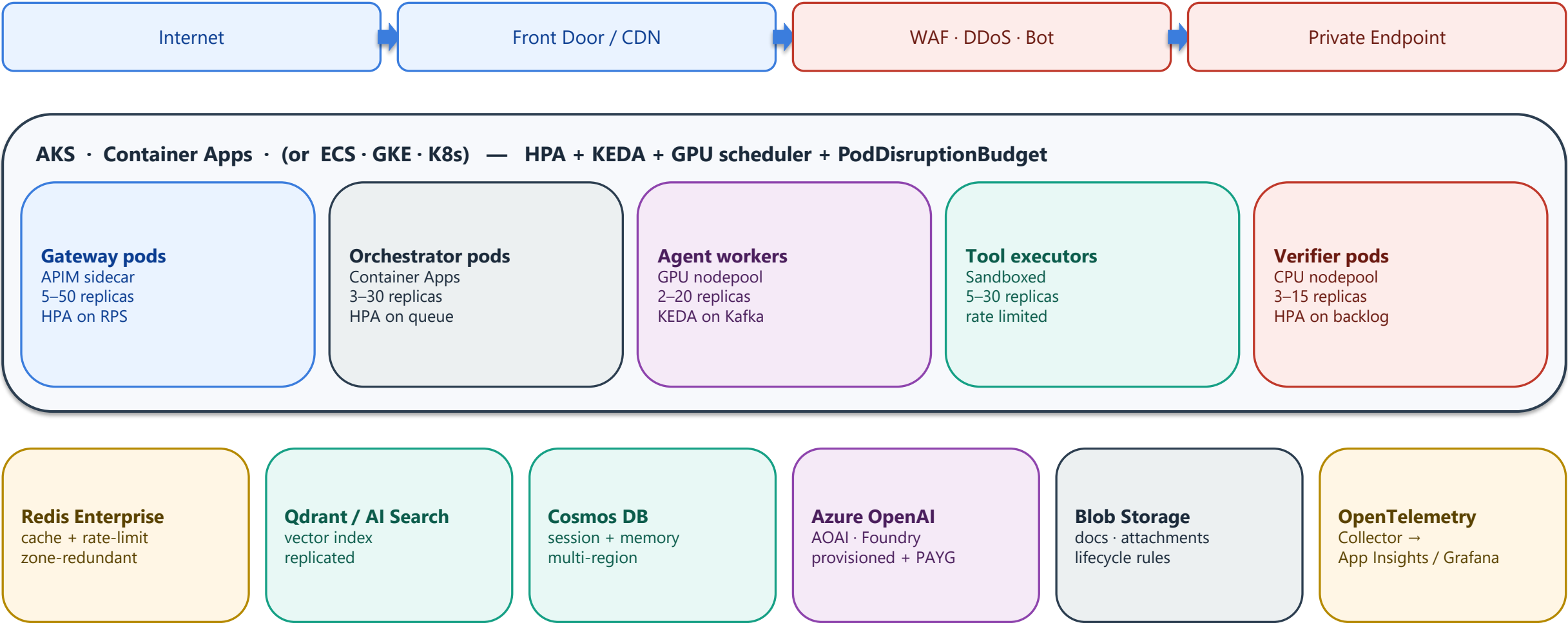
Observes · pays · improves

- CI / CD pipelines
- OpenTelemetry traces & metrics
- Cost dashboards & budgets
- Alerting & on-call
- Governance & audit logs
- Compliance evidence
- LLMOps eval & benchmarks
- Incident response
- Capacity + quota planning

Between the AI reasoning layer and cloud infrastructure sits a platform layer of shared, stateful services. Enterprise AI stands or falls on this middle.



Logical architecture ≠ deployment architecture. Here is what actually runs, autoscales and pages someone at 3 AM.



Traditional CI/CD is not enough. LLMOps adds prompt tests, eval gates, model registry, canary + shadow, cost guardrails.

Git / Repo

Prompts · agents · tools · IaC · eval sets
versioned as code

CI

Lint · sec-scan · unit · prompt tests
safety tests · eval gate

Artefact Registry

Signed containers · pinned models
prompt versions · SBOM

CD

Blue / Green · Canary · Shadow
Rollback · progressive delivery

Runtime

Continuous eval · drift · cost cap
incident + rollback triggers

Prompt lifecycle

Draft → Test → Approved → Prod → Deprecated

- Versioned prompt templates
- Prompt tests + regression suite
- Prompt registry as source of truth
- Prompt rollback independent of model

Model lifecycle

Candidate → Shadow → Canary → Prod → Retired

- Model registry (foundation + fine-tuned)
- Approved providers only (TB-C)
- Cost + latency + safety benchmarks
- Shadow traffic for regression

Tool lifecycle

Onboarded → Restricted → GA → Deprecated

- Tool registry with impact class
- Argument guardrails per tool
- HITL approval for high-impact
- Audit + rate limit per tool

Same 16 layers. Four independent mappings. Vendor-neutral by construction.

Microsoft Azure

L1 Gateway

APIM · Front Door · Entra ID

L2 Orch.

Container Apps · Durable Fn

L5 Retrieval

AI Search · Cosmos Gremlin · Fabric

L6 Models

Azure OpenAI · AI Foundry

L7 Tools

APIM · Logic Apps · MCP on ACA

L10 Obs.

App Insights · Azure Monitor

L13 Sec.

Entra · Key Vault · Purview · Defender

L14 GRC

Purview · Azure Policy · Foundry gov.

Amazon AWS

L1 Gateway

CloudFront · WAF · API Gateway · Cognito

L2 Orch.

ECS / Fargate · Step Functions

L5 Retrieval

OpenSearch · Neptune · Athena

L6 Models

Bedrock · SageMaker

L7 Tools

API Gateway · Lambda · MCP on ECS

L10 Obs.

CloudWatch · X-Ray · OTel Collector

L13 Sec.

IAM · KMS · Secrets Manager · Macie

L14 GRC

Config · Audit Manager · Bedrock
Guardrails

Google Cloud

L1 Gateway

Cloud LB · Cloud Armor · API Gateway · IAP

L2 Orch.

Cloud Run · Workflows

L5 Retrieval

Vertex AI Search · Spanner Graph

L6 Models

Vertex AI · Gemini · Model Garden

L7 Tools

API Gateway · Cloud Functions · MCP on
Run

L10 Obs.

Cloud Ops Suite · OTel Collector

L13 Sec.

IAM · KMS · Secret Manager · SCC

L14 GRC

Assured Workloads · Vertex Model
Registry

Open Source / K8s

L1 Gateway

Envoy · Kong · Keycloak · OIDC

L2 Orch.

K8s + Argo Workflows · LangGraph

L5 Retrieval

Qdrant · Weaviate · pgvector · Neo4j

L6 Models

vLLM · Ollama · TGI · TorchServe

L7 Tools

MCP servers · custom adapters

L10 Obs.

OTel · Prometheus · Grafana · Tempo · Loki

L13 Sec.

SPIFFE · Vault · Falco · Kyverno

L14 GRC

OPA / Gatekeeper · Kyverno · Sigstore

EASRA maps explicitly to the frameworks your compliance, security and risk teams already recognise.

NIST AI RMF

Govern · Map · Measure · Manage

Mapped in EASRA at: L14 Governance · L10 Observability

MITRE ATLAS

Adversary tactics + techniques for AI

Mapped in EASRA at: L1 · L8 Guardrails · L13 · L14

ISO / IEC 42001

AI management system

Mapped in EASRA at: L14 · L12 LLMOps · L15 Value

OpenTelemetry

Traces / metrics / logs schema

Mapped in EASRA at: L10 Observability across every layer

EU AI Act

Risk categorisation + provider duties

Mapped in EASRA at: L14 · L9 Verification · L13 Security

Model Context Protocol (MCP)

Standardised tool interface

Mapped in EASRA at: L7 Tooling & Actions

OWASP LLM Top 10

LLM-specific application risks

Mapped in EASRA at: L1 · L6 · L7 · L8 · L9 · L11

TOGAF

Enterprise architecture context

Mapped in EASRA at: Positions EASRA as an AI-domain reference architecture

Every consequential architectural choice in EASRA is recorded as an ADR. Not a deliverable — a first-class artefact of the standard.

ADR-001	Verification is separate from Evaluation Runtime verifiers vs. offline eval — different lifecycles, different owners.	ADR-006	Cache Manager is centralised One component for prompt · semantic · embedding · memory · response caches.
ADR-002	Memory is externalised Compute stays stateless. Sessions, profiles, episodic memory live outside.	ADR-007	Async work uses an Event Bus, not sync calls Batch, embedding, re-index, long-running agents — Kafka / Event Hub.
ADR-003	Prompt Registry is mandatory Prompts are versioned code, not literals — enables A/B, rollback, audit.	ADR-008	Control / Data / Management planes are separated Governance, runtime and operations scale independently.
ADR-004	AI Gateway is separate from API Gateway Model routing, token budgets, retries, circuit breakers ≠ HTTP concerns.	ADR-009	Standards are mapped, not replaced NIST · ISO 42001 · EU AI Act · OWASP LLM · MITRE ATLAS — complements, doesn't compete.
ADR-005	Tool Registry with impact class is mandatory Every tool has an impact class; HITL for high-impact; audit for all.	ADR-010	Dual license: CC-BY-4.0 docs · Apache-2.0 code Docs freely adaptable; code freely usable in commercial products.

ADOPT · CONTRIBUTE · FORK

EASRA is an open architecture standard.

Vendor-neutral. Standards-aligned. Verifiable. Yours to fork.

★ Star

github.com/KarthikeyanDhanakotti/EASRA

Fork & Adopt

[templates/repository-template/](#)

Contribute

[CONTRIBUTING.md](#) · [GOVERNANCE.md](#)

Follow

linkedin.com/in/karthikeyan-dhanakotti